Get Started with LLMs Course

Welcome to the **Get Started with LLMs** course.

A practical, beginner-friendly tour of today's most useful large language model tools.

Table of Contents

#	Chapter	Key Topics
1	Introduction to LLMs	LLM basics · System Prompts · Agents · Context Files · Documentation
2	ChatGPT	Prompt engineering · Best practices
3	Claude	Long context · Project workflow
4	Gemini	Multimodal AI · Google ecosystem
5	NotebookLM	Source-grounded research · Studio
6	LM Studio	Local models · curl · Python API
7	n8n	Workflow automation · Google Cloud API

© LLMs Course

Prerequisites & How to Use

Prerequisites

- A computer with internet access
- Basic familiarity with web browsers
- No prior programming experience required

How to Use This Repository

- Read all chapter learning material in this root `README.md`
- Open chapter-specific sample files inside each `examples/` folder
- Folders `01-07` are used to store chapter examples only

© LLMs Course

1. Introduction to LLMs

What is an LLM?

A **Large Language Model** is an AI model trained on very large text datasets to generate, summarize, translate, and analyze language — typically using a Transformer-based architecture.

© LLMs Course

1. Introduction — Key Concepts

- **Prompt / Response**
- **Token / Context Window**
- **Temperature**
- **System Prompt**
- **Hallucination**
- **Fine-tuning / RAG**

© LLMs Course

1. Introduction — Categories of Tools

- **Cloud-based:** ChatGPT, Claude, Gemini
- **Local / on-device:** LM Studio + open-source models
- **Workflow automation:** n8n

Example file: `01-introduction/examples/llm-glossary.md`

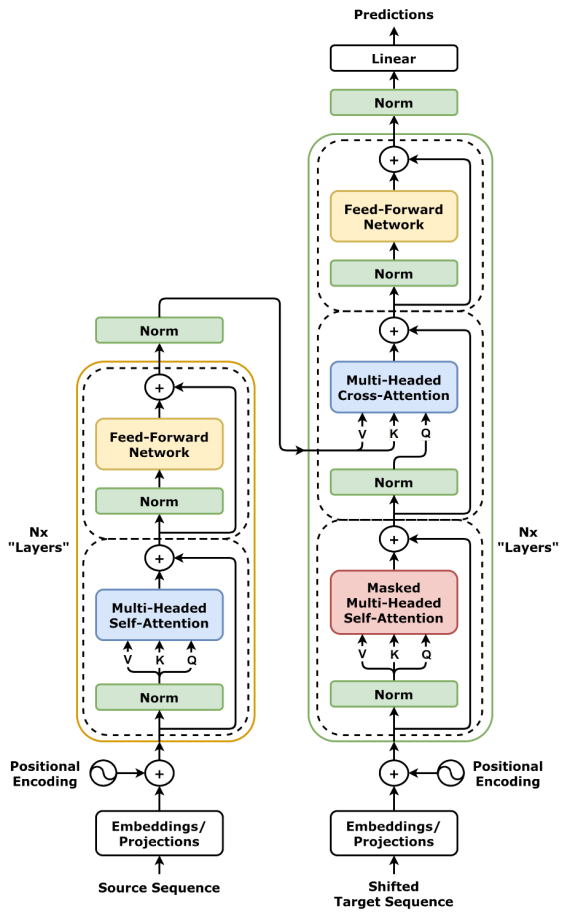
© LLMs Course

1.1 หลักการทำงานของ LLM

LLM = โมเดลทำนาย **token ถัดไป** จากบริบทที่ได้รับ

ขั้นตอนการทำงานคร่าวๆ:

1. **Tokenization** — แปลงข้อความเป็นหน่วยย่อย (tokens)
2. **Embedding** — แปลง token เป็น vector ตัวเลข
3. **Transformer + Attention** — ประมวลผลความสัมพันธ์ของ token
4. **Next-token prediction** — ทำนาย token ถัดไปทีละตัว
5. **Sampling** — สุ่มตามความน่าจะเป็น (ควบคุมด้วย temperature)



1.1 วงจรชีวิตของ LLM

ขั้นตอน	สิ่งที่เกิดขึ้น
Pre-training	เรียนรู้ภาษาจากข้อมูลขนาดใหญ่ (เว็บ, หนังสือ, โค้ด)
Fine-tuning	ปรับให้ทำตามคำสั่งเฉพาะทาง
RLHF	ใช้ feedback จากมนุษย์ปรับพฤติกรรม
Inference	ตอนที่เราเรียกใช้งานจริง (chat, API)

> LLM **ไม่ได้ "คิด" หรือ "เข้าใจ"** — มันทำนายคำถัดไปจากรูปแบบที่เคยเห็น

© LLMs Course

1.2 System Prompt ที่เปิดเผยต่อสาธารณะ

System prompt = คำสั่งซ่อนที่กำหนดบุคลิกและกฎของโมเดล

แหล่งที่เปิดเผยอย่างเป็นทางการ:

- **Anthropic** เผยแพร่ system prompt ของ Claude ที่ `docs.anthropic.com/en/release-notes/system-prompts`
- **NVIDIA** เผยแพร่ model card ของ Nemotron บน Hugging Face
- **OpenAI** ไม่เปิดเผย แต่มีหลายส่วนที่ผู้ใช้ค้นพบและบันทึกไว้

> หมายเหตุ: ใช้เพื่อ **ศึกษารูปแบบ** เท่านั้น — เนื้อหาเปลี่ยนตาม version

© LLMs Course

1.2 ตัวอย่าง: Claude Opus 4.6 / 4.7 (โครงสร้างที่ Anthropic เผยแพร่)

```
The assistant is Claude, created by Anthropic.
The current date is {{currentDateTime}}.
Claude's knowledge base was last updated in early 2025.

Claude enjoys helping humans and sees its role as...
[behavioral guidelines]

Claude uses markdown for code. Claude is now being
connected with a human.
```

องค์ประกอบหลัก: **identity → date → knowledge cutoff → behavior → tool rules → safety**

© LLMs Course

1.2 ตัวอย่าง: NVIDIA Nemotron (โครงสร้างจาก model card)

```
You are a helpful, respectful and honest assistant.
Always answer as helpfully as possible, while being safe.

If a question does not make any sense, or is not factually
coherent, explain why instead of answering something not
correct. If you don't know the answer, please don't share
false information.
```

จุดสังเกต: เน้น **"helpful, harmless, honest"** (HHH)

- instruction ป้องกัน hallucination ตรงๆ

1.3 Logic ของ LLM ที่เรียนรู้จาก system prompt

จาก system prompt เราจะเห็นว่า LLM:

1. **ทำตามคำสั่งตามลำดับ** — instruction ก่อน มีน้ำหนักกว่า
2. **ต้องการ context ชัดเจน** — บอกบทบาท, วันเวลา, knowledge cutoff
3. **ตอบดีขึ้นเมื่อมี structure** — markdown, ขั้นตอน, ตัวอย่าง
4. **ปฏิบัติตามขอบเขตที่ตั้งไว้** — "do this" / "don't do that"
5. **ไม่รู้สิ่งที่ไม่ได้บอก** — ถ้าไม่ใส่บริบท จะเดา

© LLMs Course

1.3 โครงสร้าง prompt ที่ดี (RICCE)

ส่วน	ตัวอย่าง
Role — บทบาท	"คุณเป็น product manager มือใหม่..."
Instruction — คำสั่ง	"สรุปฟีเจอร์ใหม่ใน 3 ข้อ"
Context — บริบท	"ผลิตภัณฑ์เป็น mobile app สำหรับนักเรียน"
Constraint — ขอบเขต	"ไม่เกิน 200 คำ ใช้ภาษาไทย"
Example — ตัวอย่าง	"เช่น: '🎯 ฟีเจอร์ X ช่วย...'"

© LLMs Course

1.4 เปรียบเทียบ prompt ที่ดี vs กำกวม

❌ กำกวม	✅ ดี
"เขียนอีเมลให้หน่อย"	"เขียนอีเมลขอเลื่อนนัด เหตุผลคือป่วย ภาษาทางการ ไม่เกิน 4 บรรทัด"
"ช่วยอธิบาย AI"	"อธิบาย AI ให้นักเรียน ม.4 ฟัง ใช้คำง่าย พร้อมตัวอย่าง 1 ข้อ"
"สรุปเอกสารนี้"	"สรุปเอกสารนี้เป็น 5 bullet points เน้นข้อสรุปและตัวเลขสำคัญ"
"เขียนโค้ดเรียง"	"เขียน Python function `sort_by_date(items)` รับ list ของ dict ที่มี key `date` (ISO string) คืน list ที่เรียงจากเก่าไปใหม่"

© LLMs Course

1.4 ทำไม prompt กำกวมถึงผลลัพธ์แย่

LLM จะ **เดา** ส่วนที่เราไม่ได้บอก → ผลคือ:

- 🎲 ผลลัพธ์ไม่สม่ำเสมอ (ครั้งละแบบ)
- 📏 ความยาวไม่ตรงต้องการ
- 🌐 ภาษา/โทนไม่เหมาะกับผู้ฟัง
- ❓ ไม่ได้ตอบคำถามจริงๆ

> **กฎทอง:** ถ้าเพื่อนมนุษย์ยังถามกลับ — LLM ก็จะเดา

© LLMs Course

1.5 System Prompt และหลักการสร้าง Agent

Agent = LLM + บทบาท + เครื่องมือ + เป้าหมาย

หลักการออกแบบ system prompt สำหรับ agent:

1. **กำหนดบทบาท** ชัดเจน (You are a...)
2. **ระบุความสามารถ** (You can do X, Y, Z)
3. **ระบุข้อจำกัด** (You cannot do A, B, C)
4. **กำหนดรูปแบบ output** (เช่น JSON, markdown)
5. **ระบุพฤติกรรมเมื่อไม่แน่ใจ** (ถามกลับ / บอกว่าไม่ทราบ)

© LLMs Course

1.5 ตัวอย่าง Agent System Prompt

```
คุณคือ "น้องช่วย" ผู้ช่วยตอบลูกค้าของร้านกาแฟ ABC

✅ ทำได้:
- ตอบเมนู ราคา และเวลาเปิด-ปิด
- แนะนำเครื่องดื่มตามรสชาติที่ลูกค้าชอบ
- รับ feedback และส่งต่อให้ทีม

❌ ทำไม่ได้:
- ยืนยันการสั่งซื้อ (ต้องผ่าน LINE OA)
- คืนเงิน หรือส่วนลดพิเศษ

หากไม่แน่ใจ ให้ตอบว่า "ขออนุญาตส่งต่อให้พี่ทีมงานนะคะ"
```

© LLMs Course

1.6 Context File คืออะไร

Context file = ไฟล์ที่โหลดเข้า context window อัตโนมัติ
ทุกครั้งที่เริ่มสนทนา (เช่น `CLAUDE.md`, `AGENTS.md`, `.cursorrules`)

หน้าที่:

- บอก AI ถึง **กฎเฉพาะของ project**
- ระบุ **convention การเขียนโค้ด** ที่ทีมใช้
- ให้ **context ของระบบ** (architecture, dependencies)
- ลด **hallucination** เพราะ AI มีข้อมูลจริงอ้างอิง

© LLMs Course

1.6 ตัวอย่าง Context File (CLAUDE.md)

```
# Project: E-commerce API

## Stack
- Node.js 20, TypeScript, Express, PostgreSQL

## Conventions
- ใช้ async/await ห้ามใช้ .then()
- ทุก endpoint ต้องมี Zod validation
- ทุก function ต้องมี unit test

## Don't
- อย่าใช้ `any` ใน TypeScript
- อย่า commit ไฟล์ .env
```

> AI จะอ่านไฟล์นี้ **ทุกครั้ง** ก่อนตอบ → คำตอบจะเข้ากับ project

© LLMs Course

1.7 ทำไม Documentation ถึงสำคัญในยุค AI

ในยุคก่อน AI:

- เอกสารคือคู่มือสำหรับ **มนุษย์** อ่าน
- ไม่มีก็ "อ่านโค้ดเอา"

ในยุค AI:

- เอกสารคือ **input ให้ AI** ทำงาน
- AI ดีเท่ากับเอกสารที่คุณให้
- **No docs = AI hallucinates**
- **Good docs = AI ทำงานเหมือนเพื่อนร่วมทีม**

> 💡 Documentation = เครื่องมือเพิ่ม productivity x10 ในยุค AI

1.7 สรุป: เอกสารที่ควรมีในทุก project

- `README.md` — overview, quickstart
- `CLAUDE.md` / `AGENTS.md` — กฎสำหรับ AI
- `ARCHITECTURE.md` — โครงสร้างระบบ
- `CONTRIBUTING.md` — วิธีร่วมพัฒนา
- `examples/` — ตัวอย่างจริง
- API docs (OpenAPI, JSDoc, etc.)

> เขียนเอกสารดี = ลงทุนให้ตัวเอง + AI + ทีมในอนาคต

© LLMs Course

1.8 ตารางเปรียบเทียบ ChatGPT, Claude, Gemini

	ChatGPT	Claude	Gemini
จุดแข็ง	สนทนาทั่วไป, image gen, ecosystem ใหญ่	วิเคราะห์เอกสารยาว, coding, reasoning	Multimodal, context 2M, Google ecosystem
เหมาะกับ	งานเขียน, ไอเดีย, image	งานวิเคราะห์เชิงลึก, dev work	Research, multimedia, integrate Google
ข้อควรระวัง	hallucinate ตัวเลข, ข้อมูลสด	บางครั้งระมัดระวังเกินไป	ตอบสั้น, บางครั้งขาดความลึก
ราคา	Free / Plus $20	Free / Pro $20	Free / Advanced $20

> เลือกตามงาน — ไม่มีค่ายไหนชนะทุกด้าน

© LLMs Course

2. ChatGPT

Highlights

- เริ่มต้นใช้งานผ่านเว็บได้ทันที
- เหมาะกับงานสนทนา เขียน สรุป และช่วยเขียนโค้ด
- คุณภาพผลลัพธ์ขึ้นกับคุณภาพของ prompt

© LLMs Course

2. ChatGPT — Best Practices

- ระบุเป้าหมายให้ชัดเจน
- กำหนดรูปแบบคำตอบ
- iterative prompting (ถามต่อเนื่องเพื่อปรับผลลัพธ์)
- ตรวจสอบข้อเท็จจริงที่สำคัญเสมอ

Example file: `02-chatgpt/examples/prompt-examples.md`

© LLMs Course

3. Claude

Highlights

- เด่นด้านการวิเคราะห์เอกสารยาวและการให้เหตุผล
- รองรับบริบทขนาดใหญ่ (long context)
- เหมาะกับงานวิเคราะห์เอกสาร, code review, งานเขียนเชิงลึก

© LLMs Course

3. Claude — Best Practices

- ใช้ **Projects** สำหรับงานที่ทำซ้ำ
- อัปโหลดเอกสารจริงเพื่อให้ตอบแบบ grounded
- ขอหลายแนวทางเพื่อเปรียบเทียบก่อนตัดสินใจ

Example file: `03-claude/examples/system-prompt-templates.md`

© LLMs Course

4. Gemini

Highlights

- โมเดล **multimodal** (ข้อความ / ภาพ / เสียง / วิดีโอ)
- รับ context ได้มากที่สุด (~2M tokens)
- ผสานกับ Google ecosystem (Search, Workspace)
- ใช้งานผ่าน Gemini UI หรือ API ใน Google AI Studio

© LLMs Course

4. Gemini — Best Practices

- เหมาะสำหรับ deep research และข้อมูลที่ต้องแนบ reference เพื่อตรวจสอบ
- เหมาะนำไปใช้เป็น application เฉพาะ เช่น nanobanana, stitch, canvas
- แยกทดลอง prompt ใน AI Studio ก่อนนำไปใช้จริง

Example file: `04-gemini/examples/gemini-api-examples.py`

5. NotebookLM

Highlights

- เน้นถามตอบจาก **แหล่งข้อมูลที่ผู้ใช้ให้** เป็นหลัก
- มี **citation** ช่วยตรวจสอบที่มาของคำตอบ
- เหมาะกับงาน research, study, synthesis เอกสารหลายแหล่ง

NotebookLM

© LLMs Course

5.1 แนะนำ NotebookLM เบื้องต้น

NotebookLM = AI research assistant ของ Google
ที่ทำงานบน **source ที่คุณให้เท่านั้น**

ขั้นตอนใช้งาน:

1. เข้า `notebooklm.google.com` (login Google account)
2. สร้าง **New notebook**
3. เพิ่ม **Sources** (สูงสุด 50 sources / 500K คำ ต่อ source)
4. ถามคำถามใน chat — AI จะตอบพร้อม **citation**
5. ใช้เครื่องมือ Studio เพื่อแปลง source เป็น media

© LLMs Course

5.1 ประเภท Source ที่รองรับ

ประเภท	ตัวอย่าง
เอกสาร	PDF, Google Docs, Word, txt, markdown
เว็บ	URL ของบทความ, blog, Wikipedia
วิดีโอ	YouTube link (ถอดเสียงอัตโนมัติ)
เสียง	mp3, m4a (ถอดเสียงอัตโนมัติ)
ที่วาง	Paste text ตรงๆ

> Tip: ผสมหลายประเภทใน notebook เดียวเพื่อ cross-reference

© LLMs Course

5.1 การหา Source คุณภาพ

- **Google Scholar** — งานวิจัย peer-reviewed
- **arXiv** — preprint สาย CS / ML / Physics
- **Wikipedia** — ภาพรวม + reference list
- **YouTube** — lecture, podcast, conference talks
- **GitHub** — README, docs ของ project
- **เอกสารทางการ** — government, NGO, official reports

> ตรวจ **author, date, source** ทุกครั้งก่อน import

© LLMs Course

5.1 แปลง Source เป็น Media (Studio)

NotebookLM Studio สร้าง media จาก source ได้:

Output	คำอธิบาย
🎙 **Audio Overview**	Podcast 2 host คุยกันเป็นภาษาธรรมชาติ
🧠 **Mind Map**	แผนผังความคิดเชื่อมโยงหัวข้อ
📝 **Briefing Doc**	สรุปสำหรับผู้บริหาร
❓ **FAQ**	คำถาม-คำตอบที่พบบ่อย
📚 **Study Guide**	คู่มือพร้อมคำถามทบทวน
⏱ **Timeline**	ลำดับเหตุการณ์
🎬 **Video Overview**	สไลด์พร้อม narration

© LLMs Course

5.2 Best Practices

- เริ่มจากภาพรวม (guide / summary)
- ถามแบบเจาะจงและตรวจ citation ทุกครั้ง
- ใช้การเทียบข้ามเอกสารเพื่อหาจุดร่วม / จุดต่าง

Example file: `05-notebooklm/examples/use-case-scenarios.md`

5.3 ⚠️ ข้อควรระวังของ NotebookLM

ข้อจำกัด	ผลกระทบ
ตอบจาก source **เท่านั้น**	ไม่รู้ข้อมูลนอก source ที่ใส่
คุณภาพ output = คุณภาพ source	source ผิด → คำตอบผิด
ไม่เหมาะกับ creative writing	มันคือ research tool ไม่ใช่ writer
Citation ไม่ใช่ "ความถูกต้อง"	citation บอกที่มา ไม่ได้ verify ข้อเท็จจริง
ข้อมูลถูกใช้ฝึกโมเดลได้	อย่าใส่ข้อมูลลับ / ส่วนตัว
ภาษาไทย: คุณภาพต่ำกว่า EN	บางครั้งสรุปไม่ครบ / แปลผิด

> 💡 ใช้คู่กับ **Claude / ChatGPT** สำหรับงานต่อยอด

© LLMs Course

6. LM Studio 💻

Highlights

- รัน open-source LLM บนเครื่องตนเอง (offline ได้)
- เหมาะกับงานที่ต้องการความเป็นส่วนตัว
- รองรับ local **OpenAI-compatible API**

© LLMs Course

6.1 แนะนำการใช้งาน LM Studio

ขั้นตอนเริ่มต้น:

1. ดาวน์โหลด LM Studio จาก `lmstudio.ai`
2. เปิดแอป → tab **Discover** → ค้นหาโมเดล
   เช่น `Llama 3.1 8B`, `Qwen 2.5 7B`, `Gemma 2 9B`
3. กด **Download** (เลือก quantization เช่น `Q4_K_M`)
4. ไปที่ tab **Chat** → โหลดโมเดล → เริ่มคุย
5. เปิด local server ที่ tab **Developer**
   → จะได้ endpoint `http://localhost:1234/v1`

6.1 เลือกโมเดลตามทรัพยากร

RAM / VRAM	ขนาดโมเดลที่เหมาะ
8 GB	3B - 7B (Q4)
16 GB	7B - 13B (Q4-Q5)
24 GB	13B - 32B (Q4)
32 GB+	32B - 70B (Q4)

> Q4_K_M = สมดุลระหว่างคุณภาพและขนาด (แนะนำสำหรับมือใหม่)

© LLMs Course

6.2 เรียกใช้ด้วย curl

LM Studio ใช้ API format เดียวกับ OpenAI

```
curl http://localhost:1234/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d '{
    "model": "local-model",
    "messages": [
      {"role": "system", "content": "You are a helpful assistant."},
      {"role": "user", "content": "อธิบาย LLM ใน 3 ประโยค"}
    ],
    "temperature": 0.7
  }'
```

6.2 เรียกใช้ด้วย Python (openai library)

```python
from openai import OpenAI

client = OpenAI(
    base_url="http://localhost:1234/v1",
    api_key="lm-studio"  # ใส่อะไรก็ได้ ไม่ตรวจ
)

response = client.chat.completions.create(
    model="local-model",
    messages=[
        {"role": "system", "content": "ตอบเป็นภาษาไทย"},
        {"role": "user", "content": "LLM คืออะไร?"}
    ],
    temperature=0.7,
)

print(response.choices[0].message.content)
```

6.3 Best Practices

- เริ่มจากโมเดลขนาดเล็กและ quantization สมดุล (เช่น `Q4_K_M`)
- เลือกโมเดลตามทรัพยากรเครื่อง (RAM / GPU)
- ปรับ temperature / context ให้เหมาะกับประเภทงาน

Example file: `06-lm-studio/examples/model-selection-guide.md`

7. n8n

Highlights

- สร้าง **workflow automation** แบบ visual
- ผสาน LLM กับระบบจริง (email, docs, APIs, DB)
- เหมาะกับงาน **agentic workflow** และงานที่ทำซ้ำ

n8n

7.1 แนะนำการใช้งาน n8n

n8n = visual workflow automation (open-source, self-host ได้)

ขั้นตอนเริ่มต้น:

1. ติดตั้งแบบ Cloud (`n8n.io/cloud`) หรือ self-host (Docker)
2. สร้าง **New Workflow**
3. เลือก **Trigger node** (เริ่มเมื่อไร?)
    - Manual, Schedule, Webhook, Email, etc.
4. เพิ่ม **Action nodes** (ทำอะไรต่อ?)
    - HTTP Request, Google Sheets, OpenAI, etc.
5. เชื่อม node ด้วยลากเส้น → กด **Execute Workflow**

© LLMs Course

7.1 ตัวอย่าง Workflow เบื้องต้น

```
[Schedule Trigger] (ทุก 9 โมง)
        ↓
[Google Sheets] (อ่านรายชื่อลูกค้า)
        ↓
[OpenAI / Claude] (สร้างข้อความ personalized)
        ↓
[Gmail] (ส่งอีเมล)
        ↓
[Google Sheets] (บันทึก log)
```

> Tip: ทดสอบทีละ node ด้วย **Execute step** ก่อนรันทั้ง workflow

7.2 ดึง Google Cloud API มาใช้กับ n8n

ขั้นตอนสร้าง Google credential:

1. เข้า `console.cloud.google.com` → สร้าง **Project**
2. **APIs & Services** → **Library** → เปิด API ที่ต้องการ
   (เช่น Google Sheets API, Gmail API, Drive API)
3. **Credentials** → **Create Credentials** → **OAuth client ID**
   - Application type: **Web application**
   - Authorized redirect URIs: copy จาก n8n credential page
4. ดาวน์โหลด `Client ID` + `Client Secret`
5. เพิ่มตัวเองใน **OAuth consent screen** → Test users

© LLMs Course

7.2 ตั้งค่า Credential ใน n8n

1. ใน n8n → **Credentials** → **New** → ค้นหา **Google**
2. เลือกประเภท เช่น **Google Sheets OAuth2 API**
3. ใส่ **Client ID** และ **Client Secret**
4. กด **Sign in with Google** → อนุญาต permissions
5. เก็บ credential นี้ไว้ใช้ในทุก Google node

> ⚠️ Self-host: ต้องตั้ง `WEBHOOK_URL` env เป็น public URL เพื่อให้ Google redirect กลับได้

7.2 Service Account (สำหรับ server-to-server)

ทางเลือกแทน OAuth (ไม่ต้อง login ผู้ใช้):

1. **Credentials** → **Create Credentials** → **Service Account**
2. ดาวน์โหลด JSON key
3. ใน Google Sheets / Drive: **Share** ไฟล์ให้ service account email
   (อยู่ในไฟล์ JSON: `client_email`)
4. ใน n8n: เลือก credential type **Service Account**
   → paste JSON key ทั้งไฟล์

> ใช้กับ workflow ที่รัน **อัตโนมัติ** ไม่มีคน login

7.3 Best Practices

- เริ่มจาก workflow เล็ก ๆ และทดสอบทีละ node
- ตั้ง error handling และ monitoring
- จัดการ credentials และ API limits อย่างรัดกุม

Example file: `07-n8n/examples/workflow-examples.md`

© LLMs Course

References

- OpenAI ChatGPT
- Anthropic Claude — System Prompts
- Google Gemini — AI Studio
- Google NotebookLM
- LM Studio
- n8n — Docs
- NVIDIA Nemotron (HuggingFace)

© LLMs Course

Thank You

Happy learning with LLMs! 🚀